

🎵 Examen: Venta de Entradas a Eventos con JavaScript

📋 Objetivo del Examen

En este examen práctico deberás desarrollar una **aplicación web interactiva de venta de entradas a eventos y conciertos**, conectando una interfaz frontend desarrollada con **HTML, CSS y JavaScript Vanilla** a un **servidor backend local en Node.js/Express**.

La aplicación debe permitir:

1. Consultar el catálogo de eventos y los géneros musicales disponibles desde el servidor backend local (`/api/eventos` y `/api/generos`).
2. Renderizar las tarjetas de eventos en `#eventosGrid` con nombre, artista, fecha, ubicación, precio, disponibilidad e imagen, y poblar el `<select>` de géneros.
3. Filtrar eventos por género musical con `#filtroGenero` y por nombre/artista con `#inputBusqueda` en tiempo real.
4. Implementar la compra de entradas: al hacer click en "Comprar" se abre `#modalCompra`, se calcula el total (`precio × cantidad`) en `#totalPagar` y se procesa el formulario `#formCompra` con `e.preventDefault()`.
5. Persistir el historial de compras en `localStorage` bajo la clave `'eventos_compras'`, renderizarlo en `#historialLista` y permitir su limpieza con `#btnLimpiarHistorial`.

🚩 Tabla de Entregas / Issues de GitHub

Cada entrega se corresponde con un **issue automático** en tu repositorio de GitHub. Para cerrar cada issue automáticamente, incluye el commit sugerido exacto al subir tu solución a la rama principal (`main`).

Entrega	Tarea a Realizar	Commit Sugerido
#1	Vincular <code>css/styles.css</code> y <code>js/script.js</code> en <code>index.html</code> .	<code>feat(html): vincular css y script js al html</code>
#2	Consumir la API local (<code>/api/eventos</code> y <code>/api/generos</code>) usando <code>fetch</code> y <code>async/await</code> .	<code>feat(js): consumir api de eventos con fetch y async await</code>
#3	Renderizar las tarjetas de eventos en <code>#eventosGrid</code> y poblar las opciones del filtro <code>#filtroGenero</code> .	<code>feat(js): renderizar tarjetas de eventos y filtros en el dom</code>
#4	Implementar filtrado, búsqueda, apertura del modal de compra, cálculo del total y procesamiento del formulario.	<code>feat(js): implementar filtrado compra y calculo de total</code>
#5	Persistir las compras en <code>localStorage</code> , mostrarlas en <code>#historialLista</code> y limpiarlas con <code>#btnLimpiarHistorial</code> .	<code>feat(js): persistir y gestionar historial de compras en localStorage</code>

Especificación Técnica y Requerimientos

1. Servidor Backend Local

El servidor Express provisto corre en el puerto `3000` con CORS habilitado:

- `GET http://localhost:3000/api/eventos` : Devuelve la lista completa de eventos.
- `GET http://localhost:3000/api/eventos?genero=pop` : Filtra eventos por género musical.
- `GET http://localhost:3000/api/eventos/:id` : Devuelve un evento por su ID numérico (incluye artista, invitados, tickets_disponibles, etc.).
- `GET http://localhost:3000/api/generos` : Devuelve la lista de géneros disponibles (`{ id, nombre }`).

Para iniciar el servidor backend:

```
npm start
```

2. Elementos Clave del DOM

- `#filtroGenero` : `<select>` para filtrar eventos por género musical (Pop, Rock Indie, Electrónica, etc.).
- `#inputBusqueda` : Input de texto para buscar eventos por nombre o artista en tiempo real.
- `#eventosGrid` : Contenedor grid donde se renderizan las tarjetas de eventos (`.event-card`).
- `#modalCompra` : Modal que se abre al hacer click en "Comprar" de un evento.
- `#compraContent` : Contenedor dentro del modal donde se renderiza la información del evento seleccionado (nombre, artista, fecha, precio unitario).
- `#formCompra` : Formulario con el input de cantidad y el botón de confirmar.
- `#cantidadEntradas` : Input numérico (1-10) para la cantidad de entradas a comprar.
- `#totalPagar` : Elemento donde se muestra el total calculado (`precio × cantidad`).
- `#btnConfirmar` : Botón submit del formulario para confirmar la compra.
- `#historialLista` : Lista `<ul>` donde se renderizan las compras guardadas.
- `#btnLimpiarHistorial` : Botón para vaciar el historial en `localStorage` .
- `.cerrar-modal` : Botones para cerrar el modal de compra (clase compartida).

3. Lógica de Compra

- Al hacer click en "Comprar" de una tarjeta, se consulta `/api/eventos/:id` y se muestra la info en `#modalCompra` .
- Eventos con `tickets_disponibles === 0` deben mostrar "AGOTADO" y deshabilitar el botón de compra.
- Eventos con `tickets_disponibles < 500` deben mostrar badge de "Últimas entradas".
- Al cambiar `#cantidadEntradas` , se recalcula `#totalPagar` = `precio × cantidad` .
- Al enviar `#formCompra` (`submit`), se usa `e.preventDefault()` , se registra la compra y se cierra el modal.

4. Almacenamiento Local (`localStorage`)

- **Clave obligatoria:** `'eventos_compras'`
- **Estructura:** Arreglo de objetos con `{ evento, artista, cantidad, total, fecha }` .
- Utilizar `JSON.stringify()` para guardar y `JSON.parse()` para leer.
- `#btnLimpiarHistorial` debe usar `removeItem()` o `clear()` para limpiar el historial.

Comandos de Prueba y Autoevaluación

Antes de entregar, podés autoevaluar tu trabajo localmente:

```
# Ejecutar todas las pruebas automáticas
npm test

# Ejecutar una prueba individual
npm run test:link
npm run test:fetch
npm run test:render
npm run test:events
npm run test:storage

# Validar estilo y calidad de código
npm run lint
npm run format:check
```

Instrucciones para la Ejecución Local

1. Instalar dependencias:

```
npm install
```

2. Iniciar el servidor local:

```
npm start
```

3. Abrir `index.html` en el navegador (usando la extensión **Live Server** de VS Code).
4. Abrir la consola de herramientas de desarrollador (**F12**) para verificar peticiones de red y depurar posibles errores.